Overview

This project is a production-oriented AI-powered full-stack application that allows users to submit unstructured text, interact with an AI model (LLMs) over that content, and receive structured outputs (JSON) generated by the model.

While the functional scope is intentionally kept small, the system is designed as if it were the foundation of a real product. It was designed to demonstrate how modern AI systems should be architected, keeping constraints such as cost control, security, tenant isolation, and scalability into account.

The primary use case is parsing free-form user input (food orders) into a strict schema that downstream systems can safely consume. The system is intentionally built as a stateless service so it can scale horizontally and integrate with modern frontend frameworks and cloud runtimes.

Product Philosophy & Scope Decisions

From the beginning, this project was approached as a startup-style MVP rather than a feature-complete platform. The focus was not set on UI polish or advanced AI techniques, but on building the right abstractions and the right boundaries so the system can evolve safely.

The “AI assistant that extracts structured data from free text” use case was chosen mainly due to personal interest. While the exercise did not mention a specific use case for the design, it was decided to focus the system on parsing food orders to structured data. This use case was selected because it represents a realistic production scenario where LLMs could add real value.

Several conscious scope decisions were made:

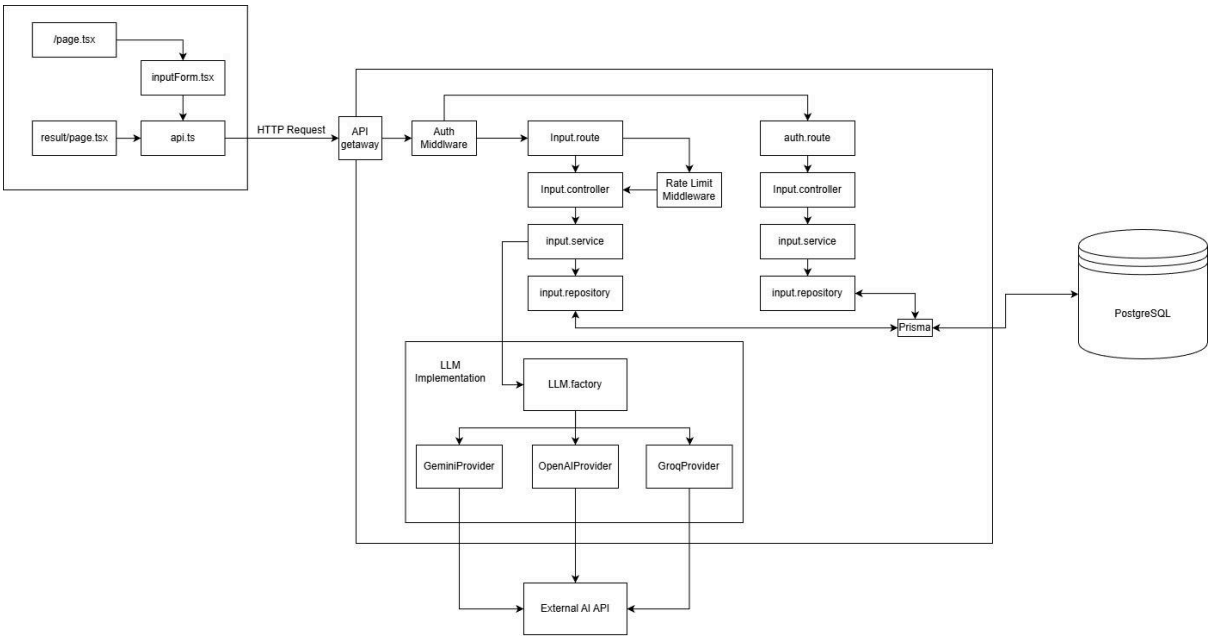
- The application does not include a traditional authentication flow (signup, login, password reset, etc.) but more of a JWT/session custom flow. This was a deliberate choice to reduce user friction and focus attention on AI system design rather than UX design, while still enabling tenant isolation and rate limits.
- The AI workflow is synchronous and request-based. A streaming workflow was implemented but not tested due to model limitations (more of this in the ‘known limitations’ section).
- Model switching is supported via environment configuration.
- This system's features include tenant isolation, rate limiting, and cost tracking.
- AI inputs and outputs are persisted for auditability, but transient metadata is minimized to reduce storage and privacy risk.

High-Level Architecture

The system is composed of a React-based frontend built with Next.js, a Node.js backend serving as an API and AI gateway, a PostgreSQL database accessed through Prisma, and an abstracted LLM provider interface featuring OpenAI, Gemini, and Groq implementations.

The frontend never communicates directly with the AI provider. All AI interactions flow through the API, which handles authentication, tenant isolation, prompt construction, rate limiting, cost tracking, and response post-processing.

The backend implemented a layer architecture and is Dockerized and deployed in AWS ECS using Fargate. The service is stateless and relies entirely on external systems (LLM providers, SSM) for state and secrets.



API Contract

Onboarding

Method: POST
Route: /api/auth
Description: Creates a new tenant if one does not already exist and issues a JWT as an HTTP-only cookie.
Payload: {}

Response: {}

AI Extraction

Method: POST
Route: /api/order
Description: Accepts unstructured text input and triggers an AI extraction.
Payload: {
mode: String [‘new’,’ refine’, ’retry’],
sessionId: String (UUID),
text: String
}
Response: {
"success": Boolean,
“Error”?: String,
"data": {
"result": {
"items": {
"name": String,
"quantity": Number,
"modifiers"?: String[]
}[],
},
"sessionId":String (UUID),
"model": String,
"timestamp": String,
"version": Number,
"id": String
"uncertainty": {
"hasPlaceholderData": Boolean
"hasGenericResponses": Boolean,
"hasEmptyItems": Boolean,
"hasSuspiciousPatterns": Boolean,
"uncertainFields": String[],
"confidenceScore": Number,
"warnings": String[],
"schemaCompliance": Number,
"retryPenalty": Number,
"tokenBehavior": Number
}
}
}

Sessions

Method: GET

Route: /api/auth
Description: Returns all extraction attempts associated with a specific session.
Payload: {}
Response: {
    data: {
        extractions{}[] //object returned in extraction
        session {
            createdAt: String
            id: String (UUID)
        }
    },
    success: Boolean
}

Key endpoints:
- `POST /api/order` — classic extraction
- `POST /api/order/stream` — SSE streaming extraction
- `GET /api/order/session/:id` — retrieve session history
- `POST /api/auth` — create tenant + session cookie

Data Model and Persistence Design

The database schema is centered around AI interactions rather than traditional user management.

Each tenant (user) owns one or more extraction sessions. A session groups multiple AI extraction attempts over time, allowing for retries, prompt versioning, and comparison between model outputs.

Each AI extraction records not only the input and structured output, but also metadata such as the model used, token counts, number of attempts, and execution status (success and failure). This information is essential for debugging, cost tracking, and future evaluation.

Usage counters and cost records are stored separately and keyed by tenant. This allows rate limiting and cost enforcement to be implemented independently from the core extraction logic.

Entities:

1. **User**
- Purpose: Stores user authentication and profile information

- Primary Key:
  - id (UUID)
- Unique Keys:
  - apikey
- Attributes:
  - id: Unique identifier (UUID)
  - apikey: Unique API key for authentication
  - createdAt: Record creation timestamp
  - updatedAt: Last update timestamp (auto-updated)

2. ExtractionSession

- Purpose: Groups related AI extractions into sessions, allows “retry” and “refine” flows without breaking auditability
- Primary Key:
  - id (UUID)
- Foreign Keys:
  - userId → User.id
- Attributes:
  - id: Unique session identifier
  - userId: Owner of the session
  - lastExtractionId: Reference to most recent extraction (nullable)
  - createdAt: Session creation timestamp

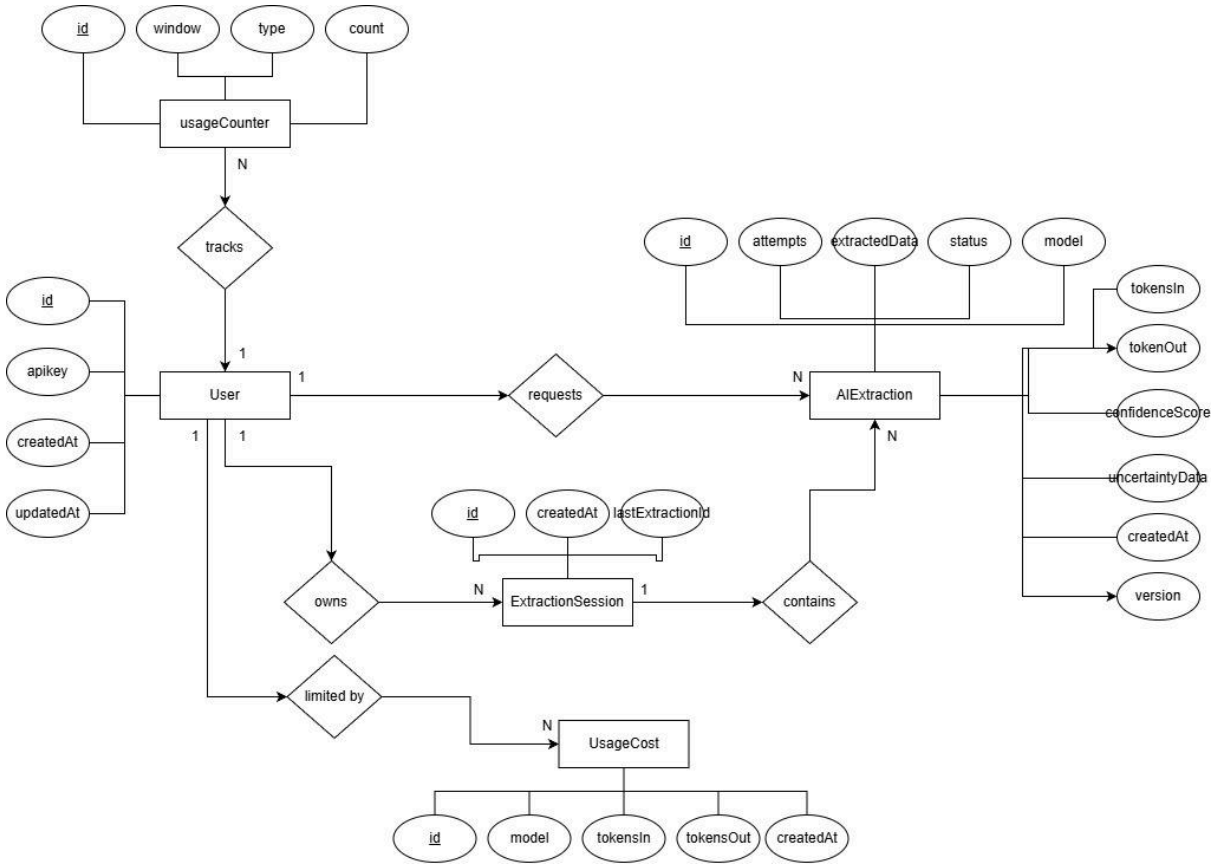
3. AIExtraction

- Purpose: Stores individual AI extraction attempts and results, supports traceability and debugging
- Primary Key:
  - id (UUID)
- Foreign Keys:
  - sessionId → ExtractionSession.id (NOT NULL)
  - userId → User.id (NULLABLE)
- Unique Constraint:
  - (sessionId, version, attempts)
- Attributes:
  - id: Unique extraction identifier
  - sessionId: Parent session reference
  - userId: User who initiated the extraction (nullable)
  - version: Version number of the extraction
  - inputText: Original text input
  - extractedData: Structured JSON output from AI
  - attempts: Number of attempts made
  - status: Enum (success/failed)
  - model: AI model used
  - tokensIn: Input token count
  - tokenOut: Output token count
  - confidenceScore: 0-100 confidence rating (nullable)
  - uncertaintyData: JSON with uncertainty flags/warnings (nullable)
  - createdAt: Extraction timestamp

4. **UsageCounter**
   - Purpose: Tracks usage metrics for rate limiting and analytics
   - Primary Key:
     - id (UUID)
   - Foreign Keys:
     - userId → User.id
   - Unique Constraint:
     - (userId, window, type)
   - Attributes:
     - id: Unique counter identifier
     - userId: User being tracked
     - window: Time window (e.g., "2026-01-28T11" for hourly)
     - type: Type of usage being counted
     - count: Current count value (default: 0)

5. **UsageCost**
   - Purpose: Records token usage and costs for billing or quota enforcement without reprocessing logs
   - Primary Key:
     - id (UUID)
   - Foreign Keys:
     - userId → User.id
   - Attributes:
     - id: Unique cost record identifier
     - userId: User incurring the cost
     - sessionId: Associated session
     - extractionId: Associated extraction
     - model: AI model used
     - tokensIn: Input tokens consumed
     - tokensOut: Output tokens generated
     - createdAt: Cost record timestamp

MER:



The system intentionally persists only data that is required for correctness, auditability, and cost control. This includes raw user input, validated structured outputs, and minimal execution metadata such as model identifiers, token counts, and status flags.

The system does not store intermediate prompt fragments, partial model outputs, or streaming tokens once a request completes. Transient runtime metadata (such as in-memory retries or intermediate validation errors) is discarded to reduce storage volume and privacy risk.

PII

The system assumes that user input may contain personally identifiable information. For this reason, data access is strictly tenant-scoped and never exposed across sessions. There is no global query path that bypasses tenant isolation.

The architecture supports future PII controls such as redaction, selective field hashing, or opt-out persistence, but these are intentionally not implemented in the MVP to avoid premature complexity.

Auditability

Auditability is achieved by persisting each AI extraction attempt as an immutable record, including input, output, prompt version, model used, and execution metadata. This allows historical decisions to be reconstructed, compared across prompt or model changes, and inspected in case of incorrect or disputed outputs.

Tenant Model and Identity Strategy

The main consideration with the identity strategy was enforcing tenant isolation while keeping onboarding friction low. This project uses an implicit authentication model that provides strong isolation without requiring explicit user authentication.

When a user first loads the application, the frontend automatically performs an onboarding request to the backend. During this process, the backend creates a new tenant record in the database and assigns it a stable internal identifier. This identifier acts as the tenant boundary for all future operations, including data access, rate limiting, and cost tracking.

At the same time, the backend issues a short-lived JWT containing only the tenant identifier and stores it as an HTTP-only cookie. From the user's perspective, there is no signup or login process. Identity is established implicitly and transparently.

AI Architecture and Design Decisions

The AI layer is explicitly designed with separation of concerns in mind.

Prompt construction is handled independently from model invocation. This allows prompts to evolve without affecting how models are called or how responses are processed.

Model invocation is abstracted behind a provider interface (Factory pattern). This makes it possible to switch LLM providers, mock responses for testing, or run experiments without touching business logic. This functionality also allows testability without paid APIs.

Response post-processing is treated as a first-class step. The system does not assume that AI output is always correct or well-formed. Outputs are validated, structured, and persisted with enough metadata to support auditing and debugging.

Prompt injection prevention

Effective prompt injection mitigation is based on three core principles: avoiding raw concatenation, restricting output formats, and limiting retry attempts.

The system implements the following defensive measures:

- Input Sanitization: User input is strictly treated as data, not executable instructions. Raw string concatenation into system prompts is prohibited.
- Structured Prompts: Clear, explicit delimiters are used to separate system instructions from user-provided content.
- Output Constraints: The LLM is explicitly instructed to produce only JSON output that conforms to a predefined schema, severely limiting its ability to execute arbitrary commands.
- Content Validation: All generated outputs are validated against a strict schema before being accepted by the system.
- Retry Limits: A maximum number of retry attempts prevents prompt injection attacks from leveraging potentially exploitable retry loops.

Prompt Design and Schema Enforcement

The extraction pipeline uses a multi-layered validation strategy and a Schema-First Architecture to ensure data integrity
The desired JSON schema is defined and version-controlled before prompting the model.

There are three types of prompts: New Extraction (Full context and schema), refinement (previous extraction + user feedback), and retry. Retry could happen in two cases: if the flow fails (schema validation, for example), in this case, the LLM is fed the specific error and asked to retry; the other situation is if the user clicks ‘retry’ and it re-attempts with a fresh context for transient failures.

Key prompt constraints are implemented, the prompt explicitly enforces tenant boundaries to prevent data leakage via the model, and instructs the LLM to output only valid JSON, matching the schema, or a structured error if compliance is not possible.

The desired JSON schema is defined and version-controlled before prompting the model, ensuring type safety, validation (Zod schemas), documentation, and systematic evolution.

Since LLMs are inherently probabilistic, the system never trusts the model output blindly. All LLM responses go through a strict validation pipeline:

1. Raw Text Parsing: Response is captured as raw text from the streaming or batch API
2. JSON Extraction: Text is parsed and converted to JSON (with error handling for malformed responses)
3. Sanitization: Removes any non-JSON artifacts (markdown code blocks, explanatory text, etc.)
4. Schema Validation: Validates against strict Zod schemas with detailed error messages
5. Business Logic Validation: Additional checks for semantic correctness (e.g., date ranges, numeric constraints)
6. Rejection on Failure: If any step fails, the output is rejected, and an error is returned to the client

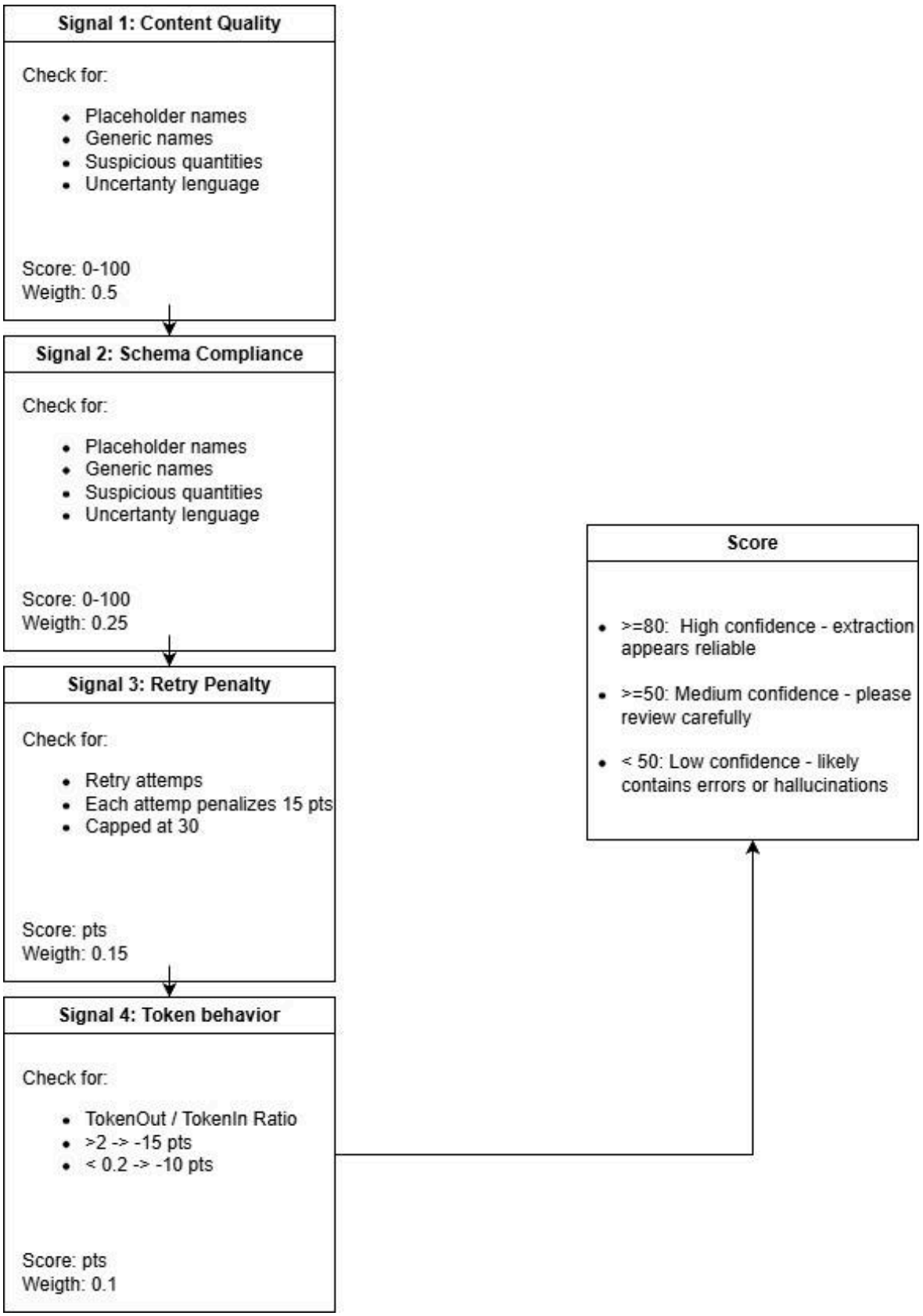
Errors definition

Validation failures are classified into actionable categories:

- Schema Mismatch: Model returned valid JSON, but structure doesn't match schema → suggests schema may need adjustment
- Parse Failure: Model returned malformed JSON → retry with clarified prompt
- Semantic Error: Data is structurally valid but semantically incorrect → return to user with guidance
- Rate Limit: Token or request limits exceeded → return 429 with retry-after header
- Model Error: Upstream LLM provider error → return 503 with generic error message

Confidence measurement

Before the response is returned to the user, it passes through a point-system confidence model. This measures content quality (if the name of an item is ‘undefined’, for example), schema compliance (check optional, but usually present, values), retry penalty (how many attempts it took the model), and token behavior ratio (if token outputs are way higher than input might indicate allucination).
While this is (intentionally) simple, it will give the users a measurable trust parameter and suggest corrective actions; it’s a good way to handle hallucinations gracefully.



Retry strategy

The AI flow supports retry logic with a maximum retry limit and structured fallback errors. In case of failure, one of two things happens: if the attempts reach the maximum limit or the failure is an LLM failure, the error gets returned. Otherwise, the prompt gets modified to make the LLM aware of the issue and resent.
While this adds LLM's calls, and consequently increases costs, it gives the user a more seamless and clean experience.

Prompt versioning

Prompt versioning is implemented at the extraction level. Each extraction attempt is associated with a specific prompt version, allowing regressions to be detected and older results to remain reproducible.

Rate Limiting and Cost Control

The system was designed and built keeping rate limiting and cost tracking as main concerns. Since (unlike traditional APIs), AI workloads are constrained by external factors (rate limiters, tokens, etc.). Consequently, scaling horizontally won't always work. The implemented solution prioritizes 'failing gracefully' under load instead of cascading failures or cost explosions.

Rate limiting is enforced per tenant rather than per IP address. This avoids common issues with IP-based limits, such as NAT sharing, VPNs, and page reload bypasses. Each tenant has a windowed usage counter that tracks request volume over time (minute, hour, and day).

Cost tracking is performed by recording token usage for each AI call. Both input and output tokens are stored, along with the model used. While token-based cost estimation is provider-dependent, it is sufficiently accurate for enforcement, monitoring, and future billing integration.

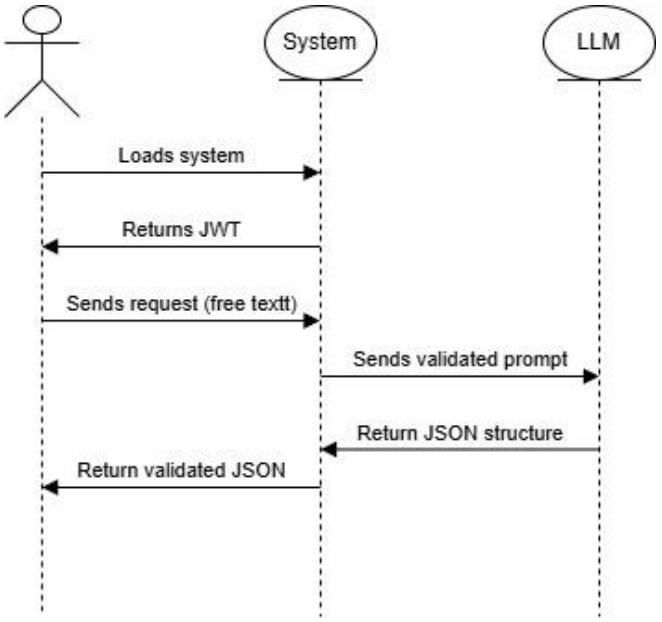
These mechanisms ensure that no single tenant can degrade system performance or incur unbounded costs, avoiding cost explosions.

Frontend Design and AI-Aware UX

The frontend is minimal (two pages) but designed with AI interaction patterns in mind.

The UI provides clear input and result pages, loading states during AI processing, and explicit error handling. The system does not attempt to hide AI uncertainty. Instead, failures and retries are surfaced clearly to the user via the uncertainty measurement system.

The frontend allows users to refine inputs and re-run extractions, reinforcing the iterative nature of AI interaction.



Infrastructure and Deployment Strategy

The backend is deployed using AWS ECS with Fargate. Infrastructure is defined using Terraform to manage the cluster, task definition, and service, ensuring repeatability and environment parity.

Secrets are stored in AWS SSM Parameter Store. ECS injects them at task startup.

Containerization

The backend is fully containerized using Docker. The image includes only compiled output and runtime dependencies, minimizing size and attack surface.

Containerization ensures consistent behavior across local development, CI, and production.

Health Check

A /health endpoint is exposed to report service liveness. This endpoint does not depend on external services and is used by ECS to determine whether a task should receive traffic.

This ensures fast recovery and zero-downtime deployments.

Secret Rotation

Secrets can be rotated by updating SSM parameters and redeploying ECS tasks (zero-downtime rotation). No code changes are required. This design supports automated rotation workflows.

Scaling under burty AI Usage

To handle spikes in requests, the service was designed to be stateless and horizontally scalable. ECS can automatically scale task count based on CPU or memory utilization. Is worth mentioning that autoscaling must be paired with concurrency limits to avoid runaway AI costs.

AI calls are isolated per request, so bursty traffic increases concurrency but does not introduce shared-state contention. Token limits ensure individual requests cannot monopolize resources.

It is worth noting that if autoescaling is enabled, a database proxy should be used to prevent connection exhaustion with the database.

Bonus Features

Streaming Responses

Streaming AI responses were implemented, though they could not be tested or properly used since Groq (the LLM API used for development) does not support streaming responses.

The implementation was designed using Server-Sent Events (SSE). Streaming is treated as a UX feature, not a correctness mechanism. While streaming is active, tokens are sent to the client for progressive rendering, while the tokens are accumulated in a buffer.

Only after the stream completes does the system attempt to parse and validate the full output. If it fails, the streamed content is discarded from a business-logic perspective, and an error is returned.

Cost Estimation

The following approximations show the cost per request, assuming typical usage of approximately 300 input and 300 output tokens:

- 1,000 requests: $\approx$ $0.23
- 10,000 requests: $\approx$ $2.25
- 100,000 requests: $\approx$ $22.50

Infrastructure expenses are typically lower than the AI costs due to horizontal scaling, and these costs are amortized.

Tenant Isolation

Multi-tenant isolation is fully implemented at both the data and middleware layers.
Multi-tenant isolation is enforced by scoping all AI prompts, sessions, and persisted data to a user identifier.

Given the current traffic assumptions and synchronous response times, introducing queues and workers would add operational complexity without actually improving user experience.
LLM tool/function-calling was also omitted since, while it could prove itself useful for this exercise, I reduced the scope to a single use case; it added, again, operational complexity unnecessarily.

Logging

Application logs are limited to operational events (request lifecycle, error conditions, rate-limit enforcement) and explicitly exclude raw AI inputs and outputs. This prevents accidental leakage of sensitive data into log aggregation systems while still enabling effective observability.

Known Limitations and Trade-Offs

Authentication & Identity
Current JWT/cookie authentication lacks password recovery and full account management. Users are logged out if browser cookies are cleared. Rate limits are bypassable via incognito/multiple browsers, requiring a future move to Redis-backed tracking.

Data Persistence
AI inputs and outputs are retained indefinitely to support debugging, auditing, and future evaluation use cases. This is a deliberate MVP trade-off.
In a production environment, retention policies would be enforced at the database level (e.g., time-based expiration per tenant or demon).

AI Provider Limitations
Streaming is implemented but not fully tested (e.g., Groq's SSE incompatibility). Advanced, provider-specific features (function calling, vision) are abstracted away. The lack of a fallback provider means primary service failure causes a complete outage.

Confidence Scoring

The confidence scoring is heuristic, not probabilistic, thus not reliably correlating with extraction correctness. The current simple point system lacks formal validation against real-world error rates.

Scalability Constraints
Synchronous request flow will introduce latency under high load. Autoscaling database management is a concern due to the absence of connection pooling guidance. Long-running extractions block the main thread due to the lack of a queue-based asynchronous processing system.

Cost Control
Token tracking is accurate, but final cost estimation is complex and provider-dependent. A major risk is the lack of hard spending caps per tenant, with only rate limits enforced. No system is in place to alert stakeholders when costs exceed thresholds.

Future Improvements

Immediate (Production-Readiness):

- Implement Redis-backed rate limiting for true multi-browser/device enforcement. This is crucial for preventing abuse and ensuring fair resource allocation across users accessing the service from different devices or sessions.
- Add data retention policies (GDPR compliance: 90-day auto-deletion). Essential for legal compliance, particularly with GDPR, by automatically purging sensitive or personal data after a set period.
- Complete health check integration in Terraform (ALB health checks). Ensures the load balancer (ALB) accurately routes traffic only to healthy instances, improving reliability and minimizing downtime.
- Implement CI/CD pipeline (GitHub Actions → ECS deployment). Automates the software release process, enabling rapid and reliable deployment of updates and new features to the Elastic Container Service (ECS) environment.

Authentication & Security:

- Replace implicit tenant model with proper OAuth2/magic link authentication. Moves away from potentially insecure implicit tenancy to a more robust, standard-based authentication mechanism (OAuth2) or a user-friendly, passwordless approach (Magic Link).
- Add API key rotation automation (AWS Secrets Manager integration). Automatically rotates sensitive API keys, reducing the risk of a compromised key being exploited long-term, managed securely via AWS Secrets Manager.
- Implement PII detection and redaction in inputs/outputs. A vital security and

compliance measure to automatically identify and mask Personally Identifiable Information (PII) processed by the system.

AI System Enhancements:

- Integrating a vector store with a RAG flow becomes particularly valuable when the system evolves beyond single-shot extraction into document-based or context-dependent use cases (questions over menu, pricing or historic data), relying solely on the LLM's static training data is insufficient and prone to hallucinations. The current architecture already supports this workflow: input embedding, vector retrieval, and controlled context injection can be added as a modular step in the prompt construction layer. This makes RAG a feasible, low-risk enhancement that significantly improves correctness and trustworthiness for document-centric scenarios without requiring a fundamental system redesign.
- Implement LLM-as-judge for automated prompt regression testing. Uses a separate Language Model (LLM) to evaluate the quality and correctness of outputs from the main model, automating the testing of changes to prompts or models.
- Multi-model ensemble (use Claude for schema generation, GPT-4 for extraction). Optimizes performance and cost by leveraging the strengths of different models for specific tasks (e.g., Anthropic's Claude for generating structured output schemas, OpenAI's GPT-4 for complex data extraction).
- Async job queue (SQS + Lambda) for long-running extractions. Decouples the request from the response for tasks that take significant time, using Amazon SQS for queuing and AWS Lambda for processing the jobs asynchronously, preventing timeouts.

Observability:

- Add structured logging (CloudWatch Logs with tenant/session correlation). Implements a standardized, machine-readable logging format, enhancing debugging and monitoring by allowing logs to be easily filtered and analyzed, crucially by correlating log entries with a specific tenant or user session.
- Model evaluation dashboard (confidence score distribution, retry rates). Provides visual insight into the AI model's performance, helping to identify potential issues or drift by tracking key metrics like the model's confidence in its outputs and how often requests fail and are retried.
- Cost analytics per tenant (spend tracking, quota enforcement). Essential for financial transparency and control, allowing the tracking of compute/API costs down to the individual tenant level, which can then be used to enforce usage quotas.

Scalability:

- ECS autoscaling policies based on queue depth (if async implemented). Dynamically scales the ECS worker services up or down based on the number of pending messages in the SQS queue, ensuring the system can handle fluctuating workloads efficiently.
- RDS Proxy to prevent connection exhaustion under load. Uses Amazon RDS Proxy to manage and pool database connections, significantly improving the application's ability to handle a large number of concurrent users without overloading the

database.
- Per-tenant quotas enforced via UsageCost table. Implements a mechanism to limit resource consumption for each tenant, based on data stored in a central UsageCost table, ensuring fair usage and preventing any single tenant from monopolizing resources.

Conclusion

This project demonstrates how an AI-powered system can be designed with production constraints in mind, even at small scale. The emphasis was placed on clear architectural boundaries, AI reliability, cost control, and tenant isolation rather than feature breadth or UI polish.

While intentionally limited in scope, the system covers end-to-end concerns, including frontend UX, backend abstraction, AI safety, persistence, and infrastructure. The design favors explicit trade-offs and predictable behavior over hidden complexity, aligning with how AI systems should be built and evolved in real production environments.

Extra comments

For the scope of this exercise, I did not expose the backend via a public endpoint.

The goal was to demonstrate infrastructure design, containerization, secure secret handling, and scalability considerations for AI workloads.

In a real production environment, this service would be exposed through an Application Load Balancer or an API Gateway, but implementing external access was intentionally deferred to keep the focus on backend architecture and operational concerns rather than networking and DNS configuration

---